After Hours Programming

Web Development Tutorials

Pythion Tutorial

Note: This is a backup copy of the tutorial at Python Programming Tutorial for Beginners - After Hours Programming. Please use this file only if there is an issue with the website. The website has the latest tutorial.

Overview

The Python tutorial is constructed to teach you the fundamentals of the Python programming language. Eventually, the Python Tutorial will explain how to construct web applications, but currently, you will learn the basics of Python offline. Python can work on the Server Side (on the server hosting the website) or on your computer.

However, Python is not strictly a web programming language. That is to say, a lot of Python programs are never intended to be used online. In this Python tutorial, we will just cover the fundamentals of Python and not the distinction of the two.

Python works much like the two previous categories, PHP and ColdFusion as they are all server side programming languages. You will see from the Python

tutorials that it's syntax is extremely different than the other two. It is probably the most clean and straightforward language you will ever learn. Just like the other languages, Python is useful because it can dynamically generate content to provide a more customized user experience. Generally, Python is a great starting language for most people, but to others, it is extremely frustrating (primarily due to spacing issues), which is why I have put the Python Tutorial at the end of the server side languages.

Enough chatter!

We want to learn Python!

Introduction

First, off **Python** usually requires some setup by downloading the Python IDLE. The Python IDLE is basically a text editor that lets you execute Python code. If you want to use Python as a server-side language, you certainly can. Python can output HTML just like other languages can, but Python is more commonly used as a module rather than intertwined like some PHP or ColdFusion. As for right now, I recommend you download the IDLE to help you debug your code while we learn the fundamentals offline. One really quick note, we are using python 3.2. Before we go to an example, please understand that Python is space sensitive. This means you must have 4 spaces for each indentation every single time. We'll get into this more later, now let's go to an example.
Example

```
print ("My first Python code!")

print ("easier than I expected")
```

ResultMy first Python Code!
easier than I expected

You can see right off the bat, that we use print() a whole lot. Basically, all it does is output whatever is inside the parentheses. You will be doing lots of

printing so, you can get more comfortable with it as we go. Print is a function that we will go into later, but just understand that it can take a value. On the first line, we provide a string value “My first Python code!”, which is a string because of the quotes. So, you just told Python to output that string to the console. Python completes that task and moves onto the next line where it prints out a different string.
See how simple that was? Well, get used to it. Python is probably one of the simplest looking languages that can do some of the most powerful things you can imagine. You can see from the example how clean Python’s syntax is without all of the extra stuff that other languages add. That covers the easiest Python statement you will ever write. In the following sections, we will be using more advanced functions and teaching you the fundamentals of Python.

Operators

With every programming language we have our **operators, and Python** is no different. Operators in a programming language are used to vaguely describe 5 different areas: arithmetic, assignment, increment/decrement, comparison, logical. There isn’t really much need to explain all of them as I have previously in the other categories, which means we will only cover a few of them.

Arithmetic Operators

Example

```
print (3 + 4)

print (3 - 4)

print (3 * 4)

print (3 / 4)

print (3 % 2)
```

```
print (3 ** 4) # 3 to the fourth power

print (3 // 4) #floor division
```

Result7
-1
12
0.75
1
81
0

See, they are pretty standard. I included how to do exponents because it's a little funky, but other than that, they are fairly normal and work as expected. Don't forget that you can use the + sign to concatenate strings. The addition, subtraction, multiplication, and division work just like expected. You might have not seen the modulus operator before (%). All the modulus operator does is to divide the left side by the right side and get the remainder. So, that remainder is what is returned and not the number of times the right number went into the left number The double * is just an easy way to provide exponents to Python. Finally, the floor division operator (//) just divides the number and rounds down.

Assignment Operators

These are pretty identical to the previous languages also, but a refresher never hurt anyone. Example please!

Example

```
a = 0

a+=2

print (a)
```

Result2

Of course, you can do this with any of the previous arithmetic operators as an assignment operator followed by the = We just told Python to add 2 to the value of a without having to say something like a = a + 2. We are programmers, and we are proud to be called lazy!

For Loop

It's time for an awesome part of Python. **Python's for loops** are pretty amazing compared to some other languages because of how versatile and simple they are. The idea of a for loop is rather simple, you will just loop through some code for a certain number of times. I won't get a chance to show you the flexibility of the for loops until we get into lists, but sure enough its time will come. Onto an example:
Example

```
for a in range(1,3):

    print (a)
```

Result

1
2

First, print is indented for a reason. Remember that Python is picky about spacing. It is somewhat complicated to understand exactly what a is in the example. In this instance, a is a variable the increments itself every time we run through the loop. Next, we use the range keyword to set the starting point and the point right after we would finish. This is precisely why the number 3 didn't print. Python is quite fond of this idea of all the way up to a number, but not including it.

Also, notice the in keyword. This is actually part of the for loop and you will understand it a bit better after we deal with lists and dictionaries. So,

basically the above for loop says, “For variable a, which will be incremented at the end of every loop, in the range of 1 up to 3.”

Classes

Well, I’m not going to lie to you. The next few tutorials are going to be rather complicated if you have never had any experience with object oriented programming (OOP). Classes are awesome because of a few reasons. First, they help you reuse code instead of duplicating the code in other places all over your program. Classes will save your life when you realize you want to change a function. You will only change it in one spot instead of 10 different spots with slightly different code. Another important part of Classes is that they allow you to create more flexible functions. First, we need to get our hands dirty and start figuring out how to make a class.

Creating Classes in Python

Example

```
#ClassOne.py

class Calculator(object):

   #define class to simulate a simple calculator

   def __init__ (self):

      #start with zero

      self.current = 0

   def add(self, amount):

      #add number to current

      self.current += amount

   def getCurrent(self):
```

```
    return self.current
```

Now, I'm not going to be like everyone else and start bombing terminology at you. Quite frankly, object oriented programming is rather difficult for a beginner and throwing these abstract concepts at you is just going to make you learn slower and hate OOP. So, a class is kind of like a function. We just establish it using the class keyword and following it up with whatever we want to name our class. In our case, we are making a calculator, so that's what I called it. How original! Next, we throw in the argument of object. This is just simply a class above this class. Don't worry much about it. It involves some serious terminology to explain. So, just type it and we will talk about it later.

Next, we get into this little guy def __init__ (self):. Abstract concept time! Objects are made from classes. So, if we had a class named cake. We could make cake objects. However, whenever you make a cake it's not the same as the previous cake you made. Sure, it may look like it and taste like it, but it's not that identical cake. So, in our class we have instances so that we know that they are two different cakes. Why do we need this you ask? Well, let's make two cakes from the same class. Now, I take a big bite out of one of the cakes. Now, you definitely want to know which cake is which right? That's why instances are so important! So, def __init__ (self): just says let's create an instance of this class. But, why do we pass in a parameter of self, you might ask. Well, it is pretty related to what we just discussed. Classes make objects and the functions in a class become the object's methods. However, we do need to know which class function belongs to which instance of the class, so we just implicitly pass in the objects property of self (discussed further in later example). Finally, we get down into the heart of the initialize function. Where we use self.current to create an instance variable equal to zero. Woo! We are done with the toughest part.

Next stop is the add function. We just pass in self and another parameter called amount. Again, self is so we know which instance. The amount is hopefully some number that was passed in. Using our previous knowledge, we understand that we are just adding whatever the amount is to the current variable.

Last, but certainly not least, we move onto this idea of "getting" variables from a class. To get the value of our self.current variable, we should use the

best practice of putting it in a function and then calling the function to get the value so that we don't mix up our instances. We set up the function and pass in the instance, and just tell Python to return the value.

Using Classes in Python

Example

```
from ClassOne import * #get classes from ClassOne file

myBuddy = Calculator() # make myBuddy into a Calculator object

myBuddy.add(2) #use myBuddy's new add method derived from the Calculator class

print(myBuddy.getCurrent()) #print myBuddy's current instance variable
```

In another file, we put in this code. Once we run it, we see it prints 2 to our screen. All of that work just to get the result of 2! Anyways, let's break this guy down. First, I have assumed that both of your classes are in the Python directory. Next, we use from ClassOne, which basically gives Python a reference to which file we are talking about. Then, we polish the statement off with import *. It just means that we want all of the classes in the file. In our case, we only have one class, the Calculator class. Next, we create an Calculator object called myBuddy by assigning it to the Calculator class with myBuddy = Calculator() . Now, this gives myBuddy all of the functions and variables in the Calculator class. (Note: once myBuddy gets those variables and functions, we call them properties and methods). Since we already initialized myBuddy, it has a property of current, and we see that property as a variable called self.current in the Calculator class that is set to 0. So, now we just call myBuddy.add(2) method, which is the add function in our Calculator class. Put simply, this is just 0 + 2. Finally, we output our instance class variable self.current by using the myBuddy.getCurrent() that returns our variable.

Well done! That was a long one. Take a breather. That's one of the toughest concepts in programming to wrap your mind around. Believe it or not, when you use strings, lists, dictionaries, etc., they all are made from classes. From here, you should take some time and study object oriented programming and attempt to apply it in Python.

Comments

Commenting in Python is also quite different than other languages, but it is pretty easy to get used to. In Python there are basically two ways to comment: single line and multiple line. Single line commenting is good for a short, quick comment (or for debugging), while the block comment is often used to describe something much more in detail or to block out an entire chunk of code.

One Line Comments

Typically, you just use the # (pound) sign to comment out everything that follows it on that line.

Example

```
print("Not a comment")

#print("Am a comment")
```

Result

Not a comment

Multiple Line Comments

Multiple line comments are slightly different. Simply use 3 single quotes before and after the part you want commented.

Example

```
'''

print("We are in a comment")

print ("We are still in a comment")
```

'''

print("We are out of the comment")

Result

We are out of the comment

Alright, we are done with comments, but don't forget them. They are your best friend in debugging complex Python code. Now onto the actual programming stuff.

Dictionaries

Python's dictionaries are not very common in other programming languages. At least at first, they don't appear normal. Some super variables like lists, arrays, etc, implicitly have an index tied to each element in them. Python's dictionary has keys, which are like these indexes, but are somewhat different (I'll highlight this in just a second). Partnered with these keys are the actual values or elements of the dictionary. Enough with my terrible explanation, let's go with an example.

Example

```
myExample = {'someItem': 2, 'otherItem': 20}

print(myExample['otherItem'])
```

Result20

See, it's a little weird isn't it? If you tried to be an overachiever, you might have tried something like print(myExample[1]). Python will bite you for this. Dictionaries aren't exactly based on an index. When I show you how to edit the dictionary, you will start to see that there is no particular order in dictionaries. You could add a key: value and, it will appear in random places.

A big caution here is that you cannot create different values with the same key. Python will just overwrite the value of the duplicate keys. With all of the warnings aside, let's add some more key:values to our myExample dictionary.
Example

```
myExample = {'someItem': 2, 'otherItem': 20}

myExample['newItem'] = 400

for a in myExample:

  print (a)
```

ResultnewItem
otherItem
someItem

Isn't that crazy how dictionaries are unordered? Now, you might not think they are unordered because my example returns alphabetical order. Well, try playing around with the key names and see if it follows your alphabetical pattern. It won't. Anyways, adding a key:value is really easy. Simply put the key in the brackets and set it equal to the value. One last important thing about dictionaries.

Example

```
myExample = {'someItem': 2, 'otherItem': 20,'newItem':400}

for a in myExample:

  print (a, myExample[a])
```

Result('newItem', 400)
('otherItem', 20)
('someItem', 2)

All we did here was to spit out our whole dictionary. In lists when we told Python to print our variable out (in our case, it's a), it would print out the value. However, with dictionaries, it will only print out the key. To get the value, you must use the key in brackets following the dictionary's name.

Dictionaries are a little confusing, but they are well worth your patience. They are lightning fast and very useful after you start using them.

Django

Now that you have studied some object oriented programming and know a good deal of Python, we should look to how you can apply this knowledge to web programming. Previously, we discussed that you could use Python to simply print the html that you wanted. Printing out your entire webpage with Python is horrendous and difficult to maintain. Sure, you might come up with some clever ways to make it more manageable, but I have a far better solution for you.

Django is an excellent web framework for Python that forces your code to be more easily maintained and has a few other perks. It helps keep your code more maintainable by separating your Python code from your html by using an MVC (model, view, and controller) pattern. Once you follow their tutorials, you'll start to understand how to separate your code into this pattern. I should mention that your typical MVC views are called templates and your typical MVC controllers are called view, which I find very confusing. But, that's the majority of my complaints with all of Django.
To be fair, learning Django isn't very easy and it has several more advanced concepts like ORM (Object-relational mapping) and it is also not the easiest thing to get running. However, as you fight your way through learning these, you will realize that they are pretty common in most other web languages. C# has entity framework, Java and ColdFusion use Hibernate, so you will eventually run into ORM. Essentially, ORM is like using your programming language to interact with a database instead of using SQL.

Anyway, please take some time and go build a website with Django. It will help you understand a fundamental paradigm of web development, MVC, and other important advanced web development concepts. Go forth and learn!

As I've said, Django is pretty difficult to learn if you are new to programming. I'd highly recommend getting a book to help you work through it, like the one below.

Editors

While I personally have a favorite Python editor, I'll try to go through several editors. I believe that someone out there should start packing Python with a better default editor as it makes coding in Python much more difficult than it should be.

At the bare minimum, you should be using Notepad++ over the default Python editor. You will still need to find a way to execute the script, but at least it has some intellisense about the language. In case you do not know what intellisense is, it is basically an aid that when you are typing code, it suggests functions, classes, etc while you are typing.

PyCharm

My personal favorite is… PyCharm! I am actually quite a fan of several JetBrains products. One benefit of using PyCharm is that it is based on a set of shortcut standards for several of their products. When you move into Java or .Net, they have editors for both (well, .NET is more like a plugin called "Resharper"). Another perk of PyCharm is that you can debug in the editor. Once you get some scripts running and start your journey into OOP (object oriented programming), try to dig into PyCharm's shortcuts and use them as much as you can. The biggest downside to PyCharm is that if you want to maintain a Django project, you must buy the professional edition. To learn more about PyCharm and how to make coding in Python more fun, take some time and study this book.

PyDev

I won't go into as much detail on the other editors as my experience with them is far less than with PyCharm. However, I do like PyDev because you can install it on Eclipse. If you haven't played with Eclipse much, it's almost like a more advanced universal editor where you can install various plugins for different languages. Obviously, this is beneficial because you only need to open one editor to work on any type of project. Anyway, PyDev has many of the great features that PyCharm has including debugging, intellisense, and more. Although, I still consider PyCharm to be the much better experience, PyDev allows Django development for free!

Other Python Editors

PyCharm and PyDev, in my opinion, are the top two editors for Python and Django. In third place, WingIDE is really powerful, but it just seems like a very old and quirky editor. Perhaps I'm shallow, but I like my editors to look as great as they feel to use.

As for other editors, I won't recommend them as I haven't found any that compete with those three. Nevertheless, the important thing about picking an editor is to make sure it feels good to you. You want an editor than makes coding both fun and productive. Have fun shopping for your new Python editor!

Editors

While I personally have a favorite Python editor, I'll try to go through several editors. I believe that someone out there should start packing Python with a better default editor as it makes coding in Python much more difficult than it should be.

At the bare minimum, you should be using Notepad++ over the default Python editor. You will still need to find a way to execute the script, but at

least it has some intellisense about the language. In case you do not know what intellisense is, it is basically an aid that when you are typing code, it suggests functions, classes, etc while you are typing.

PyCharm

My personal favorite is… PyCharm! I am actually quite a fan of several JetBrains products. One benefit of using PyCharm is that it is based on a set of shortcut standards for several of their products. When you move into Java or .Net, they have editors for both (well, .NET is more like a plugin called "Resharper"). Another perk of PyCharm is that you can debug in the editor. Once you get some scripts running and start your journey into OOP (object oriented programming), try to dig into PyCharm's shortcuts and use them as much as you can. The biggest downside to PyCharm is that if you want to maintain a Django project, you must buy the professional edition. To learn more about PyCharm and how to make coding in Python more fun, take some time and study this book.

PyDev

I won't go into as much detail on the other editors as my experience with them is far less than with PyCharm. However, I do like PyDev because you can install it on Eclipse. If you haven't played with Eclipse much, it's almost like a more advanced universal editor where you can install various plugins for different languages. Obviously, this is beneficial because you only need to open one editor to work on any type of project. Anyway, PyDev has many of the great features that PyCharm has including debugging, intellisense, and more. Although, I still consider PyCharm to be the much better experience, PyDev allows Django development for free!

Other Python Editors

PyCharm and PyDev, in my opinion, are the top two editors for Python and Django. In third place, WingIDE is really powerful, but it just seems like a very

old and quirky editor. Perhaps I'm shallow, but I like my editors to look as great as they feel to use.

As for other editors, I won't recommend them as I haven't found any that compete with those three. Nevertheless, the important thing about picking an editor is to make sure it feels good to you. You want an editor than makes coding both fun and productive. Have fun shopping for your new Python editor!

Formatting

We waited a little bit to talk about formatting because it might get a little intense with how much you can do and how easily you can do things with **variables in Python**. Formatting in Python is a little bit odd at first. But, once you get accustomed to it, you'll be glad you did.

Formatting Numbers as Strings

Example

```
print('The order total comes to %f' % 123.44)

print('The order total comes to %.2f' % 123.444)
```

ResultThe order total comes to 123.440000
The order total comes to 123.44

Ya, I told you it's a little weird. The f following the first % is short for float here because we have floating numbers and Python has a specific way of dealing with formatting decimals. The left % tells Python where you want to put the formatted string. The value following the right % is the value that we want to format. So, Python reads through the string until it gets to the first % then Python stops and jumps to the next %. Python takes the value following the second % and formats it according to the first %. Finally, Python places that second value where the first % is. We can use a single value such as a string

or a number. We can also use a tuple of values or a dictionary. Alright, this is great, but what about formatting strings?

Formatting Strings

Strings are just like how we were formatting the numbers above except we will use a s for string instead of an f like before. Usually, you will only want to format a string to limit the number of characters. Let's see it in action:
Example

```
a ="abcdefghijklmnopqrstuvwxyz"

print('%.20s' % a)
```

Resultabcdefghijklmnopqrst

Functions

Functions in Python are super useful in separating your code, but they don't stop there. Any code that you think you will ever use anywhere else, you should probably put in a function. You can also add arguments to your functions to have more flexible functions, but we will get to that in a little bit. You can define a function by using the keyword def before your function name. Let's use our first Python function.
Example

```
def someFunction():

  print("boo")

someFunction()
```

Resultboo

It is necessary to define functions before they are called. Even though we have to skip over the function while reading to see the first statement, someFunction(). This throws us back up into the def someFunction():, again with a colon following it. Then after we acknowledge

that the function is being called, we create a new line with four spaces for our simple print statement.

Functions with Arguments

Simple functions like the one above are great and can be used quite often. However, there usually comes a time where we want to have the function act on data that the user inputs. We can do this with arguments inside of the () the follows the function name.

Example

```
def someFunction(a, b):

  print(a+b)

someFunction(12,451)
```

Result463

Using the statement someFunction(12,451), we pass in 12, which becomes a in our function, and 451, which becomes b. Then, we just have a little print statement that adds them and prints them out.

Function Scope

Python does support global variables without you having to explicitly express that they are global variables. It's much easier just to show rather than explain:

Example

```
def someFunction():

  a = 10

someFunction()

print (a)
```

This will cause an error because our variable, a, is in the local scope of someFunction. So, when we try to print a, Python will bite and say a isn't defined. Technically, it is defined, but just not in the global scope. Now, let's look at an example that works.
Example

```
a = 10

def someFunction():

  print (a)

someFunction()
```

In this example, we defined a in the global scope. This means that we can call it or edit it from anywhere, including inside functions. However, you cannot declare a variable inside a function, local scope, to be used outside in a global scope.

IF Statements

In the heart of programming logic, we have the **if statement in Python**. The if statement is a conditional that, when it is satisfied, activates some part of code. Often partnered with the if statement are else if and else. However, Python's else if is shortened into elif. If statements are generally coupled with variables to produce more dynamic content. Let's cut to an example really quick.
Example

```
a = 20

if a >= 22:

  print("if")

elif a >= 21:

  print("elif")
```

```
else:

  print("else")
```

Resultelse

So, we have the variable a that equals twenty. Now, we run it through our if statement that checks to see if a is greater than or equal to 22. It isn't. So, we skip past the inner print statement and continue to the elif statement. This conditional checks if a is greater than or equal to 21. It isn't either, which brings us to the final leg of our expanded if statement. The else is the catch all, which means if the previous conditionals are not satisfied, we will run the code inside it. So, we run the print("else"), and we see in the results, the string else.

The If Syntax

The most important thing you might have missed in the example above is how Python's syntax is a lot cleaner than other languages. However, this also means it is very picky and tends to bite beginners with errors. After every conditional we have a colon. Next, you must proceed to a new line with 4 spaces to tell Python you only want this code with 4 spaces to be run when the previous conditional is satisfied. Ok, well it doesn't have to be 4 spaces, but you must be consistent with the spaces you use to indent. You can use 3 every time if you want, but 4 is kind of a standard. Also, if you wanted to run more than one statement after the conditional is satisfied, you must have a new line with the same number of spaces before the next statement.

Lists

We don't have arrays; we have **Python lists**. Lists are super dynamic because they allow you to store more than one "variable" inside of them. Lists have

methods that allow you to manipulate the values inside them. There is actually quite a bit to show you here so let's get to it.

Example

```
sampleList = [1,2,3,4,5,6,7,8]

print (sampleList[1])
```

Result2

The brackets are just an indication for the index number. Like most programming languages, Python's index starting from 0. So, in this example 1 is the second number in the list. Of course, this is a list of numbers, but you could also do a list of strings, or even mix and match if you really wanted to (not the best idea though). Alright, now let's see if we can print out the whole list.

Example

```
sampleList = [1,2,3,4,5,6,7,8]

for a in sampleList:

  print (a)
```

Result1

2
3
4
5
6
7
8

I told you we would come back to see how awesome the for loop is. Basically, variable a is the actual element in the list. We are incrementing an implicit index. Don't get too worried about it. Just remember we are cycling through the list.

Common List Methods

There are number of methods for lists, but we will at least cover how to add and delete items from them. All of the list methods can be found on Python's documentation website. Methods follow the list name. In the statement listName.append(2), append() is the method.

- .append(value) – appends element to end of the list
- .count('x') – counts the number of occurrences of 'x' in the list
- .index('x') – returns the index of 'x' in the list
- .insert('y','x') – inserts 'x' at location 'y'
- .pop() – returns last element then removes it from the list
- .remove('x') – finds and removes first 'x' from list
- .reverse() – reverses the elements in the list
- .sort() – sorts the list alphabetically in ascending order, or numerical in ascending order

Try playing around with a few of the methods to get a feel for lists. They are fairly straightforward, but they are very crucial to understanding how to harness the power of Python.

Reading Files

Many times, a programmer finds a reason to read content from a file. It could be that we want to read from a text file, such as a log file, or an XML file for some serious data retrieval. Sometimes, it is a massive task to figure out how to do it exactly. No worries, Python is smooth like always and makes reading files a piece of cake. There are primarily 2 ways in which Python likes to read. To keep things simple, we are just going to read from text files, feel free to explore XML on your own later. XML is a pretty cool markup, but as you get deeper, it is kind of a headache. Enough of my tangents, to the coding board:

Opening a File in Python

test.txt

I am a test file.

Maybe someday, he will promote me to a real file.

Man, I long to be a real file

and hang out with all my new real file friends.

Example

```
f = open("test.txt", "r") #opens file with name of "test.txt"
```

This is pretty simple to explain. We have our awesome little test.txt file filled with some random text. Now, in the example we have our code. Just like you click a file to open it, Python needs to know what file to open. So, we use the open method to tell Python what we want to open and to go ahead and open (make a connection to) it. The “r” just tells Python that we want to read (it is “w” when we want to write to a file). And, of course, we set this new connection to a variable so we can use it later. However, we have only opened a file, which is not all that exciting. Let’s try reading from the file.

Python File Reading Methods

- file.read(n) – This method reads n number of characters from the file, or if n is blank it reads the entire file.
- file.readline(n) – This method reads an entire line from the text file.

Example

```
f = open("test.txt","r") #opens file with name of "test.txt"

print(f.read(1))

print(f.read())
```

ResultI
am a test file.
Maybe someday, he will promote me to a real file.
Man, I long to be a real file
and hang out with all my new real file friends.

Woot woot! So, this might be a little confusing. First, we open the file like expected. Next, we use the read(1), and notice that we provided an argument of 1, which just means we want to read the next character. So, Python prints out “I” for us because that is the first character in the test.txt file. What happens next is a little funky. We tell Python to read the entire file with read() because we did not provide any arguments. But, it doesn’t include that “I” that we just read!? It is because Python just picks up where it left off. So, if Python already read “I”, it will start reading from the next character. The reason for this is so you can cycle through a file without have to skip so many characters each time you want to read a new character. It’s complicated, I know, but think of reading like a one way process. Python is lazy, it doesn’t want to go back and reread contents. Let’s take this one step further with reading lines.

Example

```
f = open("test.txt","r") #opens file with name of "test.txt"

print(f.readline())

print(f.readline())
```

ResultI am a test file.
Maybe someday, he will promote me to a real file.

Ha! This time we were prepared for Python’s trickery. Since we just used the readline() method twice, we knew that we would get first 2 lines because of Python’s reading process. Of course, we also knew that readline() reads only a line because of it’s simple syntax. Alright, one last important, complicated, and awesome thing about Python’s reading abilities.

Example

```
f = open("test.txt","r") #opens file with name of "test.txt"

myList = []

for line in f:

   myList.append(line)

print(myList)
```

Result[‘I am a test file.\n’, ‘Maybe someday, he will promote me to a real file.\n’, ‘Man, I long to be a real file\n’, ‘and hang out with all my new real file friends.’]

What!? Python just blew our minds! First, don’t freak out with the \n. It is just a newline character, since those lines are on a different file, Python wants to keep that format (you can always strip them out later).We open the file like normal, and we create a list, which we have mastered by now. Then, we break the file into lines in our for loop using the in keyword. Python is magically smart enough to understand that it should break files into lines. Next, as we are looping through each line of the file we use myList.append(line) to add each line to our myList list. Finally, when we print it out, Python shows us its glory. It broke each line of the file into a string, which we can manipulate to do whatever we want.

Wait! Don’t forget to close the file!

Example

```
f = open("test.txt","r") #opens file with name of "test.txt"

print(f.read(1))

print(f.read())

f.close()
```

The important thing I saved until the end is that you should always close your files using the close() method. Python gets tired running around opening files and reading them, so give it a break and close the file to end the connection. It is always good practice to close files, your memory will thank you. Next, let’s do some construction by writing to files.

Strings

As with any programming language, strings are one of the most important things to know about Python. Also as we have experienced in the other

languages so far, strings contain characters. Strings are not picky by any means. They can contain almost anything if used properly. The are also not picky by the amount of characters you put in them. Quick Example!

Example

```
myString = ""

print (type(myString))
```

Result<class 'str'>

The type() is awesome. It returns the variable type of whatever is inside the parentheses, which is very useful if you have a few bugs that you can't figure out or if you haven't looked a large chunk of code for awhile and don't know what type the variable is. Back to the amount of characters in a string, we can see even an empty set of "" returns as a string. Strings are powerful and are very easy to declare. Let's look into some common string methods so you can get your hands dirty.

Common String Methods in Python

- stringVar.count('x') – counts the number of occurrences of 'x' in stringVar
- stringVar.find('x') – returns the position of character 'x'
- stringVar.lower() – returns the stringVar in lowercase (this is temporary)
- stringVar.upper() – returns the stringVar in uppercase (this is temporary)
- stringVar.replace('a', 'b') – replaces all occurrences of a with b in the string
- stringVar.strip() – removes leading/trailing white space from string

String Indexes

One of the really cool things about Python is that almost everything can be broken down by index and strings are no different. With strings, the index is

actually the character. You can grab just one character or you can specify a range of characters.

Example

```
a = "string"

print (a[1:3])

print (a[:-1])
```

Resulttr
strin

Let's discuss the print (a[1:3]) because it is the easiest to explain. Remember that Python starts all indexes from 0, which would have been the 's' in our variable a. So, we print out 'tr' because we printed everything up to our range of 3, but not 3 itself. As for the second example, welcome to a beautiful part of Python. Essentially, specifying a negative number after a : in an index means that you want python to calculate the index starting from the end and moving toward the front aka backwards. So, we tell python that we want everything from the first character to the second to last character. Take a breather, you earned it.

Tuples

In Python, tuples are almost identical to lists. So, why should we use them you ask? The one main difference between tuples and lists are that tuples cannot be changed. That is to say you cannot add, change, or delete elements from the tuple. Tuples might seem odd at first, but there is a great reason behind them being immutable. As programmers, we mess up occasionally. We change variables that we didn't want to change, and sometimes, well, we just want things to be constant so we don't accidentally change them later. However, if we change our minds we can also convert tuples into lists or lists into tuples. The fact is we need to make the conscious

effort to say Python, I want to change this tuple into a list so I can modify it. Enough babbling, let's see a tuple in action!

List vs. Tuple

Example

```
myList = [1,2,3]

myList.append(4)

print (myList)



myTuple = (1,2,3)

print (myTuple)



myTuple2 = (1,2,3)

myTuple2.append(4)

print (myTuple2)
```

Result[1, 2, 3, 4]
(1, 2, 3)
Traceback (most recent call last):
File "C:/Python32/test", line 9, in
myTuple2.append(4)
AttributeError: 'tuple' object has no attribute 'append'

So, we see that the list clearly works as expected. We just append a 4 onto the end, and Python doesn't miss a beat. Next, we test out our tuple declaration and it works as well. But when we try to append to the tuple, Python give us a nasty little error. Like I said, you cannot change a tuple! Python will bite you if you try using things like append on a tuple. But, let's say "Hey, I really want to add a four to that tuple." Let's do it:

Example

```
myTuple = (1,2,3)

myList = list(myTuple)

myList.append(4)

print (myList)
```

Result[1, 2, 3, 4]

Boom! We have successfully undone what Python was trying to teach us not to do. We just casted the tuple into a list, then once it was a list, we used it's append method to add the 4. It should be reiterated that the purpose of a tuple is to be immutable. If you are planning to change the variable, just use a list instead.

Variables

In the heart of every good programming language, including **Python, are the variables**. Variables are an awesome asset to the dynamic world of the web. Variables alone are dynamic by nature. However, Python is pretty smart when it comes to variables, but it is sometimes quite a pain. Python interprets and declares variables for you when you set them equal to a value. Example:

Example

```
a = 0

b = 2

print(a  + b)
```

Result

2

Isn't that cool how Python figured out how a and b were both numbers. Now, let's try it with the mindset of wanting the 0 to be a string and the 2 to also be a string to create a new string "02".

Example

```
a = "0"

b = "2"

print(a  + b)
```

Result

02

Ah, see! Now, it thinks they are both strings simply because we put them in quotations. Don't worry if you don't understand strings yet, just note that they are not numbers or integers. This is really awesome, but as with any element or practice in a programming language there is always the flip side, especially with this auto declaration thing. Suppose we started off with "0" being a string, but we change our mind and want it to be a number. Finally, we decide that we want to add it to the number 2. Python bites us with an error. This brings us to the idea of casting the variable into a certain type.

Example

```
a = "0"

b = 2

print(int(a)  + b)
```

Result

2

Whoa, what is that int() thing? This is how you cast one variable type into another. In this example, we cast the string 0 to an integer 0. Let's take a quick peak at a few of the important variable types in Python:

- int(variable) – casts variable to integer
- str(variable) – casts variable to string

- float(variable) – casts variable to float (number with decimal)

Like I said, those are just the most commonly used types of casting. Most of the other variable types you typically wouldn't want to cast to.

While Loop

While loops in Python can be extremely similar to the for loop if you really wanted them to be. Essentially, they both loop through for a given number of times, but a while loop can be more vague (I'll discuss this a little bit later). Generally, in a while loop you will have a conditional followed by some statements and then increment the variable in the condition. Let's take a peek at a while loop really quick:

Example

```
a = 1

while a < 10:

  print (a)

  a+=1
```

Result1
2
3
4
5
6
7
8
9

Fairly straightforward here, we have our conditional of a < 10 and a was previously declared and set equal to 1. So, our first item printed out was 1, which makes sense. Next, we increment a and ran the loop again. Of course, once a becomes equal to 10, we will no longer run through the loop.

The awesome part of a while loop is the fact that you can set it to a condition that is always satisfied like 1==1, which means the code will run forever! Why is that so cool? It's awesome because you create listeners or even games. Be warned though, you are creating an infinite loop, which will make any normal programmer very nervous.

Where's the do-while loop

Simple answer, it isn't in Python. You need to consider this before you are writing your loops. Not that a do-while loop is commonly used anyway, but Python doesn't have any support for it currently.

Writing to Files

Reading files is cool and all, but writing to files is a whole lot more fun. It also should instill a sense of danger in you because you can overwrite content and lose everything in just a moment. Despite the threat of danger, we press on. Python makes writing to files very simple. With somewhat similar methods to reading, writing has primarily 2 methods for writing. Let's get to it!

Warning! The Evil "w" in the Open Method

Example

```
f = open("test.txt","w") #opens file with name of "test.txt"

f.close()
```

...And whoops! There goes our content. As I said, we are in dangerous water here friends. As soon as you place "w" as the second argument, you are basically telling Python to nuke the current file. Now that we have nuked our file, let's try rebuilding it.

Example

```
f = open("test.txt","w") #opens file with name of "test.txt"

f.write("I am a test file.")

f.write("Maybe someday, he will promote me to a real file.")

f.write("Man, I long to be a real file")

f.write("and hang out with all my new real file friends.")

f.close()
```

If you were continuing from the last tutorial, we just rewrote the contents that we deleted to the file. However, you might be flipping out screaming, “but it’s not the same!” You are 100% correct my friend. Hit the control key a couple of times and cool off, I’ll show you the fix in a minute. Ultimately, the write() method is really easy. You just pass a string into it (or a string variable) and it will write it to the file following that one way process it does. We also noticed that it kept writing without using any line breaks. Let’s use another method to fix this.

Writing Lines to a File

We have the a fairly easy solution of just putting a new line character “\n” at the end of each string like this:

Example

```
f.write("Maybe someday, he will promote me to a real file.\n")
```

It’s just a simple text formatting character. Yes, even text files have a special formatting similar to how HTML documents have their own special formatting. Text files are just much more limited than HTML.

Appending to a File

Example

```
f = open("test.txt","a") #opens file with name of "test.txt"

f.write("and can I get some pickles on that")

f.close()
```

Boom! While our text file makes absolutely no sense to a human, we both know we just had a big victory. The only big change here is in the open() method. We now have an “a” (for append) instead of the “w”. Appending is really just that simple. Now go out there and write all over the world my friend.